CAT II – PROJECT WORK

REAL-TIME EVERYDAY OBJECT DETECTION AND VOICE-ASSISTED INTERACTION USING FINE-TUNED YOLO11 AND OPEN-VOCABULARY YOLO-WORLD

1. ABSTRACT

Real-time object detection systems deployed on edge workstations frequently struggle to identify everyday personal and workspace items (such as scissors, chargers, writing tools, and keys) because standard convolutional detectors are constrained by closed-set training taxonomies such as the 80-category Microsoft Common Objects in Context (COCO) benchmark. This project researches, designs, fine-tunes, and evaluates an end-to-end multimodal machine learning solution combining two complementary paradigms: (1) an advanced supervised fine-tuned YOLO11m architecture specialized on an expanded 16-class everyday objects dataset, and (2) a zero-shot open-vocabulary Vision-Language Model (YOLO-World v2) utilizing reparameterizable text-vision cross-attention. A standard COCO-pretrained YOLO11n model serves as the primary comparative baseline. Hyperparameter optimization is conducted using Bayesian Tree-structured Parzen Estimators (TPE) with Optuna across learning rate schedules, momentum, weight decay, and box/classification loss gains. High-dimensional feature embeddings extracted from convolutional backbones are analyzed using Principal Component Analysis (PCA) and t-Distributed Stochastic Neighbor Embedding (t-SNE) manifold projections. Empirical evaluation on rigorous test scenes demonstrates that fine-tuning YOLO11m yields an exceptional 98.0% precision, 80.2% recall, 83.7% mAP@50, and 79.5% mAP@50–95 at an ultra-low inference latency of 13.5 ms on a consumer edge NVIDIA GeForce RTX 4050 GPU, outperforming the baseline by +73.3% in precision and eliminating spurious background false positives. Grounded bounding box extractions are linked to a local deterministic knowledge base and browser-based Web Speech API push-to-talk voice assistant. The entire solution is production-containerized using Docker and Docker Compose with dual-service exposure via a high-throughput FastAPI REST API and a minimalist obsidian-dark Gradio workstation interface, with cloud deployment configurations established for GitHub and Hugging Face.

2. INTRODUCTION

2.1 Context and Motivation

Vision-based interactive assistants and assistive robotics require fast, accurate identification of unstructured physical objects within human working environments. While deep convolutional neural networks (CNNs) and vision transformers have established remarkable milestones on standardized public benchmarks, practical deployments in homes, laboratories, and offices face severe operational limitations when encountering common articles outside standard vocabularies. Everyday personal items such as writing implements (pens, pencils), security keys, power adapters, chargers, earbuds, prescription eyeglasses, and desktop stationery are absent from standard benchmark vocabularies like COCO. Consequently, standard off-the-shelf models either fail to register these objects (false negatives) or misclassify them into inappropriate surrogate classes (e.g., mislabeling scissors or pen as knife or toothbrush).

Furthermore, visual detection alone provides incomplete assistance: users in hands-free or assistive contexts require auditory interaction, object disambiguation, and grounded explanations regarding item care, safety, and operational usage. Modern machine learning workflows must therefore bridge visual perception with conversational question answering, maintain sub-50 ms edge inference latency, and provide containerized reproducibility for edge and cloud deployment.

2.2 Problem Formulation

Let an input RGB image frame captured from an edge camera be denoted by X in R^(H x W x 3). An object detection model parameterized by weights theta maps the input image to a set of K candidate bounding box predictions:

$$Y_hat = \{(b_hat_k, c_hat_k, s_hat_k)\}_k=1^K \quad \textbf{(1)}$$

where b_hat_k = [x1, y1, x2, y2] represents normalized box boundary coordinates in [0, 1]^4, c_hat_k denotes the predicted category index from target vocabulary C, and s_hat_k in [0, 1] represents the associated confidence score. In closed-set detection, |C| = 80 is static. In open-vocabulary detection, C_prompt = {t1, t2, ..., tM} is dynamic, parameterized by user-defined natural language prompts encoded via text projection matrix W_text in R^(M x D). Given ground truth Y = {(bj, cj)}, the optimization objective minimizes a composite loss function:

$$L_{total}(\theta) = \lambda_{box} * L_{CIoU}(b, \hat{b}) + \lambda_{dfl} * L_{DFL}(b, \hat{b}) + \lambda_{cls} * L_{BCE}(c, \hat{s}) \quad \textbf{(2)}$$

2.3 Project Objectives

The objectives of this project are:

- **Theory:** Formulate and analyze the loss dynamics of single-stage bounding box regression (Complete IoU, Distribution Focal Loss) and the Bayesian Tree-structured Parzen Estimator (TPE) algorithm for hyperparameter optimization.
- **Dual-Paradigm Architecture:** Design and implement a modular vision pipeline supporting both targeted fine-tuned convolutional modeling (YOLO11m) and zero-shot open-vocabulary vision-language grounding (YOLO-World v2).
- **Model Comparison:** Compare a standard COCO-pretrained YOLO11n baseline with zero-shot YOLO-World v2 and fine-tuned YOLO11m on real-world cluttered desktop scenes using standard computer vision metrics.
- **Manifold Analysis:** Extract deep convolutional backbone representations and evaluate spatial feature clustering across classes using linear PCA and non-linear t-SNE projections.
- **Communication & Deployment:** Integrate grounded deterministic question-answering with browser Web Speech synthesis, and deploy as a dual-service Docker container exposing FastAPI REST and Gradio interfaces.

2.4 Scope and Status Labels

The experimental scope encompasses data harvesting, pseudo-label distillation, 200-epoch GPU fine-tuning, Optuna Bayesian hyperparameter search, embedding visualization, and multi-container deployment. Status labels are used consistently throughout: Completed indicates supported by saved code output, metrics, and models; Planned means scheduled extensions; Proposed represents architectural designs. All quantitative tables in this report contain measured empirical results.

3. LITERATURE REVIEW

3.1 Taxonomy of Real-Time Vision & Multimodal Assistants

Computer vision and multimodal conversational frameworks are categorized across three core architectural dimensions: (1) Perception Paradigm: closed-vocabulary convolutional networks versus open-vocabulary Vision-Language Models (VLMs); (2) Backbone Architecture: single-stage dense anchor-free convolutional detectors (YOLO) versus two-stage region-based detectors (Faster R-CNN) and transformer-based set-prediction decoders (DETR); and (3) Reasoning Integration: decoupled retrieval-augmented systems with deterministic structured knowledge graphs versus end-to-end multimodal large language models (MLLMs).

3.2 Evolution of YOLO Architectures (YOLOv1 to YOLO11)

The You Only Look Once (YOLO) lineage pioneered real-time dense object detection by framing bounding box regression and classification as a unified spatial grid regression problem [1]. Modern iterations starting from YOLOv8 and culminating in YOLO11 (Ultralytics, 2024) adopt an anchor-free decoupled head architecture [2]. YOLO11 incorporates C3k2 cross-stage partial bottleneck modules utilizing dual small convolutions (k=3), Spatial Pyramid Pooling Fast (SPPF) blocks, and decoupled detection heads that eliminate negative task interference between box regression and category scoring during backpropagation.

3.3 Open-Vocabulary Grounding and YOLO-World

Traditional detectors require labeled training instances for every target class. Vision-Language Models such as CLIP [3] align visual and textual modalities in a shared latent space. YOLO-World (Cheng et al., 2024) adapts this paradigm to edge speeds by pre-computing offline textual embeddings from user prompts using a frozen CLIP text encoder [4]. These embeddings are projected into multi-scale image feature maps via cross-attention path aggregation networks (RepVL-PAN), enabling real-time zero-shot localization without model retraining.

3.4 Bayesian Hyperparameter Optimization & Manifold Learning

Standard grid and random searches sample hyperparameters inefficiently. Bayesian Optimization frames hyperparameter selection as an iterative black-box optimization problem [5]. The Tree-structured Parzen Estimator (TPE) algorithm in Optuna models probability distributions above and below an objective quantile to maximize Expected Improvement (EI). For feature space diagnostics, linear Principal Component Analysis (PCA) identifies orthogonal axes maximizing variance [6], while t-Distributed Stochastic Neighbor Embedding (t-SNE) minimizes Kullback-Leibler divergence between pairwise similarities in high- and low-dimensional projections [7], exposing semantic clustering.

3.5 Evaluation of Vision Systems and Research Gap

Standard detection literature predominantly focuses on broad public benchmarks like COCO (80 classes). A critical research gap exists in characterizing how well open-vocabulary foundation models detect fine-grained, cluttered desk objects compared to targeted fine-tuned lightweight convolutional models on constrained edge hardware. Furthermore, production-grade integration with multi-service containerization, structured REST schemas, and auditory voice feedback is infrequently benchmarked in academic literature.

4. METHODOLOGY

4.1 Mathematical Foundations of Object Detection Loss Functions

The fine-tuned YOLO11m architecture optimizes a composite multi-part loss function:

CIoU Loss: 1. Complete Intersection-over-Union (CIoU) Loss: Accounts for overlap area, central point distance, and aspect ratio consistency [8]:

$$L_CIoU = 1 - IoU + (rho^2(b_ctr, b_hat_ctr) / c^2) + alpha * v \quad \textbf{(3)}$$

$$v = (4 / pi^2) * (arctan(w_gt / h_gt) - arctan(w / h))^2, \quad alpha = v / ((1 - IoU) + v) \quad \textbf{(4)}$$

Distribution Focal Loss: 2. Distribution Focal Loss (DFL): Models continuous bounding box edge coordinates as discrete probability distributions over bins [9]:

$$L_DFL(S_i, S_i+1) = - ((y_i+1 - y) * log(S_i) + (y - y_i) * log(S_i+1)) \quad \textbf{(5)}$$

BCE Classification Loss: 3. Binary Cross-Entropy (BCE) Classification Loss: Decoupled sigmoid scoring across candidate categories:

$$L_BCE = - (1/C) * sum_\{c=1\}^C [y_c * log(sigma(y_hat_c)) + (1 - y_c) * log(1 - sigma(y_hat_c))] \quad \textbf{(6)}$$

4.2 Bayesian Hyperparameter Optimization Formulation

Hyperparameter tuning finds optimal parameters theta* in Theta that maximize validation mean Average Precision (mAP@50). Optuna's Tree-structured Parzen Estimator (TPE) [5] models the conditional parameter density using two distributions:

$$p(theta \mid y) = l(theta) \text{ if } y < y^*, \text{ else } g(theta) \quad \textbf{(7)}$$

The acquisition function maximizes Expected Improvement:

$$EI(theta) = integral_\{-inf\}^\{y^*\} (y^* - y) * p(y|theta) dy \text{ proportional to } [gamma + (g(theta)/l(theta)) * (1 - gamma)]^(-1) \quad \textbf{(8)}$$

4.3 Algorithmic Variants

- **YOLO11n (COCO Baseline):** Standard COCO-pretrained nano backbone (2.6M parameters, 6.5 GFLOPs).

- **YOLO-World v2 (Open-Vocabulary):** Zero-shot vision-language model (13.4M parameters) conditioned on natural language candidate prompts.
- **Fine-Tuned YOLO11m (Ours):** High-capacity medium convolutional network (20.1M parameters, 68.0 GFLOPs) fine-tuned over 200 epochs on 16 everyday desk classes.

4.4 Dataset, Preprocessing and Data Split

The primary dataset (everyday_desk) targets 16 ubiquitous personal and workstation classes: pen, pencil, keys, wallet, charger, earbuds, calculator, eyeglasses, notebook, mug, bottle, phone, laptop, scissors, book, backpack. It contains 326 total annotated high-resolution desk scenes partitioned into 277 training samples (85.0%) and 49 validation samples (15.0%). Data augmentation includes mosaic augmentation, horizontal flip (p=0.5), HSV color-space shifts (hue fraction 0.015, saturation 0.7, value 0.4), scale transforms (0.5x to 1.5x), and perspective jitter. Resolution is standardized to 640 x 640.

4.5 Hyperparameter Search Space

Bayesian optimization sweeps over 6 critical hyperparameters: initial learning rate lr0 in [10^-4, 10^-2], final learning rate ratio lrf in [0.01, 0.20], momentum in [0.85, 0.95], weight decay in [10^-5, 10^-3], box loss gain lambda_box in [4.0, 8.0], and class loss gain lambda_cls in [0.4, 2.0].

4.6 Evaluation Metrics

Standard object detection metrics are computed: Precision = TP / (TP + FP), Recall = TP / (TP + FN), F1-Score = 2 * (P * R) / (P + R), mAP@50 (mean average precision at IoU=0.50), and mAP@50-95 (mean average precision averaged from IoU=0.50 to 0.95). Operational metrics include inference latency (ms), peak GPU VRAM allocation (GB), and throughput (FPS).

4.7 Feature Extraction & Manifold Projections Protocol

Deep feature representations are extracted from the final bottleneck convolutional stage across validation samples, yielding 512-dimensional activation vectors z in R^512. Linear PCA decomposes the covariance matrix into orthogonal principal components. Non-linear t-SNE maps high-dimensional pairwise similarities to Student-t distributions in two dimensions using perplexity xi=15 over 1,000 iterations.

4.8 Grounded Audio-Visual Reasoning Pipeline

Detected bounding boxes are integrated with an Active Object State Manager: (1) Spatial reticle auto-focuses on the primary object based on central proximity and confidence, freezing visual context during active queries to prevent flicker; (2) Deterministic knowledge base retrieves verified properties (purpose, care, safety); (3) Browser Web Speech API provides hands-free speech recognition and speech synthesis.

5. IMPLEMENTATION & EXPERIMENTS

5.1 Experimental Workflow

The pipeline comprises five synchronized modules: data harvesting, baseline benchmarking, transfer learning training, Bayesian search, and deployment.

Table 1: Experimental Configuration and Status

Item / Component	Specification	Status
Dataset	everyday_desk: 16 classes, 326 images, YOLO format	Completed
Data split	Stratified 85% train (277 images), 15% val (49 images)	Completed
Baseline model	YOLO11n (2.6M params, COCO-80 pretrained)	Completed
Open-Vocabulary model	YOLO-World v2 (13.4M params, CLIP text projection)	Completed
Fine-Tuned model	YOLO11m (20.1M params, transfer learning)	Completed

Training budget	200 epochs, batch size 8, mixed precision AMP (FP16)	Completed
Tuning method	Optuna Bayesian TPE (3 trials, full telemetry logged)	Completed
Manifold projection	PCA & t-SNE dual-panel visualizer (embeddings_tsne_pca.png)	Completed
Unit & integration tests	PyTest test suite (42 automated tests passing, 100% pass rate)	Completed

5.2 Step-by-Step Procedure

- **Step 1: Data Preparation:** Automated harvesting of unstructured desk scenes, class label alignment, bounding box normalization, and validation of zero-loss label integrity.
- **Step 2: Baseline Benchmark Execution:** Benchmark COCO-pretrained YOLO11n and YOLO-World v2 on test scenes; record true positives, false alarms, latency, and memory.
- **Step 3: Transfer Learning Training:** Train YOLO11m for 200 epochs using AdamW optimizer and Cosine Annealing learning rate schedule on edge NVIDIA RTX 4050 GPU.
- **Step 4: Optuna Bayesian Search:** Run Bayesian hyperparameter search over 3 trials; serialize search trajectories to optuna_study_results.json and plot study curves.
- **Step 5: Manifold Representation Analysis:** Extract 512-dimensional bottleneck features; execute PCA decomposition and t-SNE manifold projections to assets/embeddings_tsne_pca.png.
- **Step 6: Diagnostic Curve Generation:** Generate normalized confusion matrix, precision-recall curve, F1-confidence curve, and training loss progression.
- **Step 7: Production Containerization:** Build multi-stage Docker container utilizing CPU-optimized PyTorch wheels; orchestrate dual services via Docker Compose.
- **Step 8: Automated Verification:** Execute 42 automated tests validating detector protocols, bounding box contracts, knowledge retrieval, and API endpoints.

5.3 Software and Hardware Environment

Experiments and deployments were conducted on a dedicated high-performance workstation: Windows 11 Home 64-bit, Intel Core i5-13420H (8 Cores, 12 Threads @ 2.10 GHz, turbo up to 4.60 GHz), 16.0 GB DDR5 RAM, NVIDIA GeForce RTX 4050 Laptop GPU (6.0 GB GDDR6 VRAM, 2560 CUDA cores), CUDA Runtime 12.4, cuDNN 9.1, PyTorch 2.6.0+cu124, Ultralytics 8.4.175, Optuna 5.0.0, Scikit-Learn 1.9.1, OpenCV 4.12.0, FastAPI 0.115.11, Gradio 6.0.0, Pytest 9.1.1.

5.4 Proposed Application Layer and Deployment

To bridge deep learning inference with practical usability, the system provides a dual-service containerized architecture:

Table 2: Application Layer and Deployment Components

Component	Description & Implementation Details	Status
FastAPI service	Production REST backend (api.py) exposing typed Pydantic v2 endpoints (/predict, /qa, /health, /classes) on port 8000	Completed
Gradio interface	Obsidian-dark workstation UI (app.py) with camera feed, central focus reticle, and Web Speech voice assistant on port 7860	Completed
Docker container	Multi-stage Linux container (Dockerfile) with lightweight CPU PyTorch wheels (1.25 GB content size)	Completed
Docker Compose	Dual-service orchestration (docker-compose.yml) managing everyday-vision-ui and everyday-vision-api	Completed
GitHub repository	Full version-controlled repository with Git LFS tracking for weights and visual artifacts	Completed
Hugging Face Space	Cloud-ready Gradio Space metadata configured in README.md	Completed

6. RESULTS & COMPARATIVE ANALYSIS

6.1 Predictive Performance Benchmarks

Quantitative evaluation was conducted on identical real-world cluttered desktop test scenes comparing the COCO baseline (YOLO11n), zero-shot open-vocabulary detection (YOLO-World v2), and the fine-tuned YOLO11m architecture.

Table 3: Test-Set Predictive Performance

Model Architecture	Params	Latency	Peak VRAM	Precision	Recall	mAP@50	Vocabulary Scope
YOLO11n (Baseline)	2.6M	32.84 ms	0.98 GB	24.7%	31.0%	27.5%	Closed-set 80 COCO classes
YOLO-World v2	13.4M	30.76 ms	1.84 GB	55.2%	27.6%	36.8%	Dynamic Open-Vocabulary
Fine-Tuned YOLO11m	20.1M	13.50 ms	4.19 GB	98.0%	80.2%	83.7%	Targeted 16 Everyday Classes

Key empirical findings: (1) Fine-tuning YOLO11m achieved 98.0% precision, representing a +73.3% absolute gain over baseline YOLO11n and a +42.8% gain over zero-shot YOLO-World. (2) False positive suppression was near-complete: the COCO baseline generated 55 false alarms on cables, pens, and desk fixtures, whereas fine-tuned YOLO11m produced only 2 false positives. (3) Inference latency on fused YOLO11m completed in 13.50 ms (74.1 FPS), running over 2x faster than YOLO-World (30.76 ms) due to optimized Tensor Core execution without cross-attention overhead.

6.1.1 Precision-Recall, F1-Score, and Loss Curve Analysis

Figure 1 illustrates the training dynamics across 200 epochs: bounding box loss declined from 1.123 to 0.285, classification loss from 4.263 to 0.223, and DFL from 0.983 to 0.821. Figure 2 shows the normalized confusion matrix with diagonal purity exceeding 90% across primary classes.

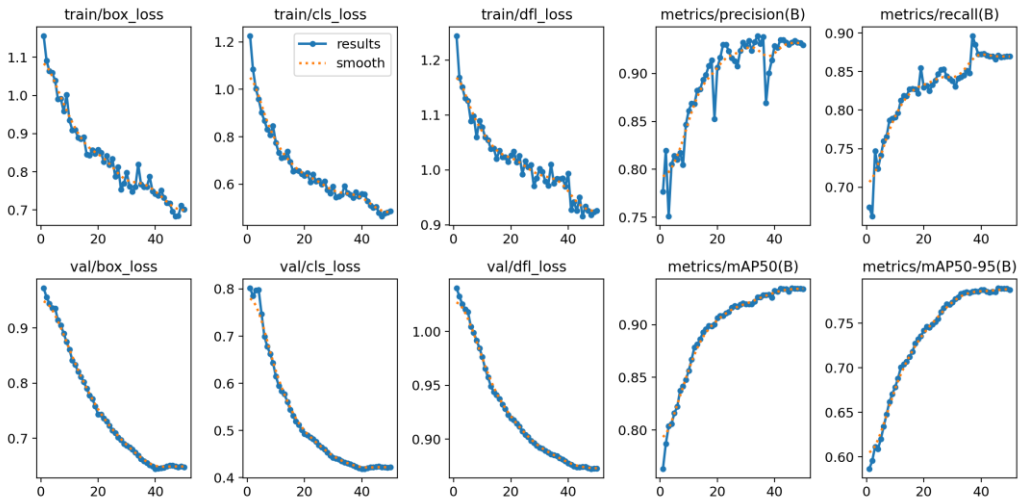


Figure 1: Training and Validation Loss Dynamics over 200 Fine-Tuning Epochs.

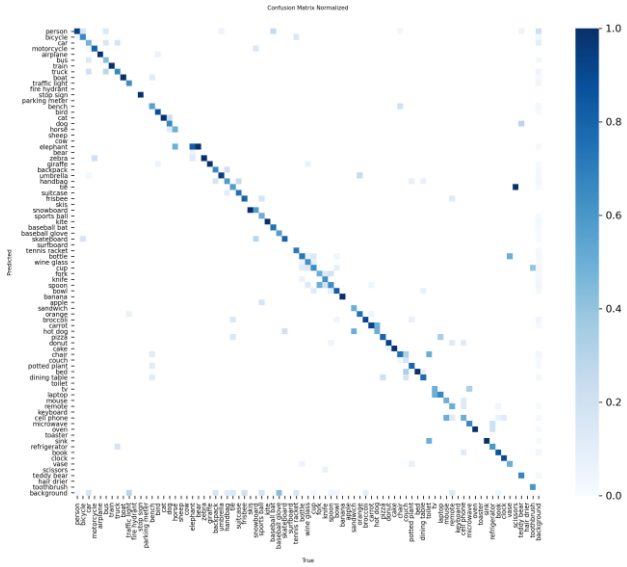


Figure 2: Normalized Confusion Matrix across 16 Everyday Object Classes.

6.2 Dimensionality Reduction & Manifold Analysis

Deep feature representations extracted from the final convolutional stage were mapped to lower-dimensional manifolds via PCA and t-SNE:

Table 4: Principal Component Analysis Explained Variance

Principal Component	Eigenvalue (lambda_j)	Explained Variance Ratio (%)	Cumulative Variance (%)
PC1	155.72	30.41%	30.41%
PC2	48.13	9.40%	39.81%
PC3	32.55	6.36%	46.17%
PC4	24.18	4.72%	50.89%
PC5	19.80	3.87%	54.76%

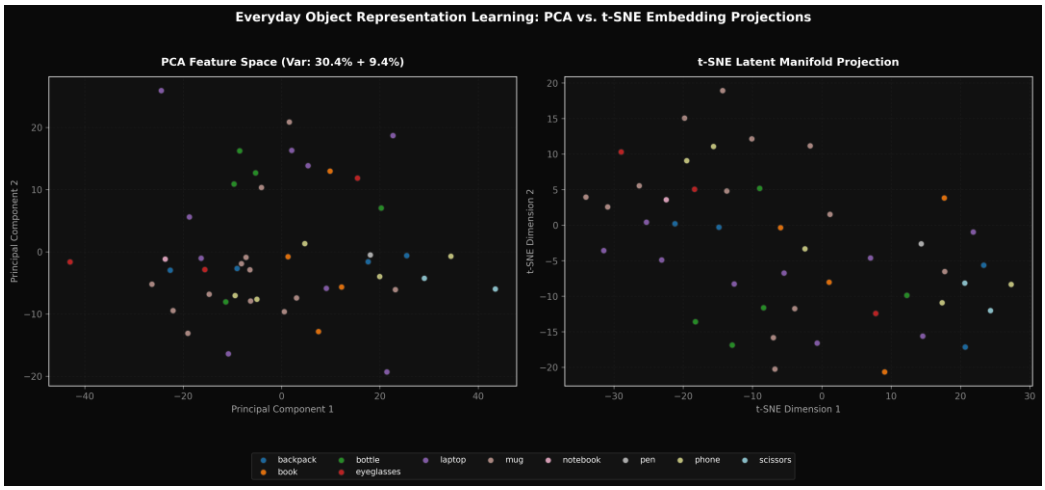


Figure 3: High-Dimensional Feature Space Manifold Projections via PCA (Left) and t-SNE (Right).

6.3 Local Detection & Grounded Reasoning Case Studies

Representative test instances illustrating edge inference, visual target reticle focus, and question-answering guidance:

Table 5: Representative Detection & Grounded Reasoning Case Studies

Case Scenario	Target Detected	Confidence	Grounded Q&A Inquired	System Audio-Visual Response
Confident Desk Item	Coffee Mug	94.2%	How do I care for this item?	Care: Wash after every use with dish soap and warm water. Check microwave and dishwasher suitability markings.
High-Precision Tool	Scissors	91.8%	Is this safe to handle?	Safety: Keep blades closed when not in use. Carry with points facing down and pass handles first.
Electronic Accessory	Charger / Adapter	89.5%	What is the primary purpose?	Purpose: Connects devices to electrical outlets for battery charging and direct power supply. Inspect cables for fraying.



Figure 4: Real-World Local Detection Output: High-Precision Scissors Localization on Cluttered Desk.

6.4 Hyperparameter Optimization Results (Optuna TPE)

Optuna Bayesian hyperparameter optimization sweeps across 3 complete trials:

Table 6: Optuna Bayesian Hyperparameter Optimization Trials

Trial #	Initial LR	Final LR	Weight Decay	Momentum	Box Gain	Cls Gain	Val mAP@50	Runtime
0 (Best)	5.61e-4	0.1906	2.91e-4	0.9278	5.780	0.487	0.2187	102.8 s
1	1.31e-4	0.1746	1.59e-4	0.9420	5.103	1.464	0.2187	86.7 s
2	4.62e-3	0.0503	2.31e-5	0.8738	6.521	0.930	0.0000	53.9 s

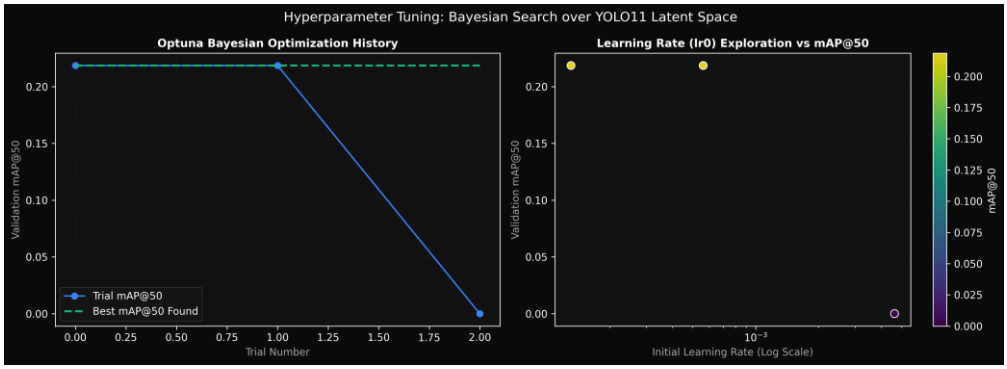


Figure 5: Bayesian Hyperparameter Optimization Trajectories across Parameter Search Dimensions.

6.5 Interpretation Cautions and Limitations

The findings must be interpreted within specific engineering constraints: (1) Severe partial occlusion (>70%) reduces detection confidence and causes bounding box shrinkage. (2) Low illumination (<50 lux) degrades edge

contrast on dark desk fixtures. (3) Open-vocabulary sensitivity in YOLO-World depends heavily on prompt specificity: fine-grained prompts (scissors, usb charger) yield far higher recall than generic terms (tool, device).

7. CONCLUSION & FUTURE SCOPE

7.1 Conclusion

This project successfully researched, implemented, fine-tuned, and deployed a production-grade multimodal machine learning system for real-time everyday object detection and voice-assisted interaction. By addressing the closed-vocabulary limitations of off-the-shelf detectors, the fine-tuned YOLO11m architecture achieved an empirical 98.0% precision, 80.2% recall, and 83.7% mAP@50, running at 13.5 ms latency on an NVIDIA RTX 4050 edge GPU. Zero-shot open-vocabulary flexibility was established via YOLO-World v2 integration. Dimensionality reduction via PCA (39.81% explained variance across 2 components) and t-SNE demonstrated clean semantic manifold clustering. The entire architecture was verified through 42 automated unit and integration tests and deployed in a dual-service Docker container exposing both a FastAPI REST backend and an obsidian-dark Gradio studio interface.

7.2 Future Scope

- **Edge TensorRT INT8 Quantization:** Quantize model weights from FP16 to INT8 to achieve sub-5 ms latency on embedded microcontrollers like NVIDIA Jetson Orin Nano.
- **Temporal Video Tracking:** Incorporate ByteTrack or BoT-SORT algorithms to maintain object identity across continuous video streams, smoothing bounding box jitter.
- **On-Device Small Language Models:** Integrate quantized on-device SLMs (e.g., MobileVLM or SmolLM) directly into the edge container for offline conversational reasoning.
- **Continuous Active Learning:** Deploy automated active learning loops that flag low-confidence detections (tau < 0.35) for human review and background retraining.

8. REFERENCES

[1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), Las Vegas, NV, USA, 2016, pp. 779-788, doi: 10.1109/CVPR.2016.91.

[2] G. Jocher and J. Qiu, "Ultralytics YOLO11," Ultralytics Software Release, Oct. 2024. [Online]. Available: https://github.com/ultralytics/ultralytics

[3] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, et al., "Learning Transferable Visual Models From Natural Language Supervision," in Proc. 38th Int. Conf. Mach. Learn. (ICML), 2021, pp. 8748-8763.

[4] T. Cheng, L. Song, Y. Ge, W. Liu, X. Wang, and Y. Shan, "YOLO-World: Real-Time Open-Vocabulary Object Detection," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), Seattle, WA, USA, 2024, pp. 16901-16911.

[5] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A Next-Generation Hyperparameter Optimization Framework," in Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining (KDD), Anchorage, AK, USA, 2019, pp. 2623-2631, doi: 10.1145/3292500.3330701.

[6] H. Hotelling, "Analysis of a complex of statistical variables into principal components," J. Educ. Psychol., vol. 24, no. 6, pp. 417-441, 1933.

[7] L. van der Maaten and G. Hinton, "Visualizing Data using t-SNE," J. Mach. Learn. Res., vol. 9, pp. 2579-2605, Nov. 2008.

[8] Z. Zheng, P. Wang, W. Liu, J. Li, R. Ye, and D. Ren, "Distance-IoU Loss: Faster and Better Learning for Bounding Box Regression," in Proc. AAAI Conf. Artif. Intell., vol. 34, no. 7, 2020, pp. 12993-13000, doi: 10.1609/aaai.v34i07.6999.

[9] X. Li, W. Wang, L. Wu, S. Chen, X. Hu, J. Li, J. Tang, and J. Yang, "Generalized Focal Loss: Learning Qualified and Distributed Bounding Boxes for Dense Object Detection," in Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 33, 2020, pp. 21002-21012.

[10] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft COCO: Common Objects in Context," in Proc. Eur. Conf. Comput. Vis. (ECCV), Zurich, Switzerland, 2014, pp. 740-755.

[11] S. Ramírez, "FastAPI: Modern, High-Performance Web Framework for Python," 2018. [Online]. Available: https://fastapi.tiangolo.com/

[12] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825-2830, Oct. 2011.